

UNIVERSIDAD PANAMERICANA

Campus México

Materia: Sistemas Operativos

Profesor: Iram Maximiliano Lopez Guerrero

Fecha de entrega: Final de semestre — Mayo 2026

Ciclo: Enero 2026

"Proyecto de Sistemas Operativos"

Miembros del Equipo		
ID	Nombre (ap.Paterno, ap.Materno, nombre)	Carrera
0246717	Flores Alcocer Andoreni Claudio	LIDCI

Mini escáner de red inspirado en nmap implementado en C++ sobre macOS

1. Resumen

El proyecto consiste en el desarrollo de una herramienta de escaneo de red inspirada en nmap, implementada en C++ sobre un sistema tipo macOS. La aplicación permite realizar un escaneo de puertos TCP sobre una dirección IP, un nombre de host o un rango de IPv4, clasificando el estado de cada puerto como **OPEN**, **CLOSED** o **FILTERED**, y opcionalmente leyendo el banner inicial del servicio asociado.

El proyecto está dirigido a administradores de sistemas, estudiantes y profesionales de ciberseguridad que necesitan una herramienta ligera para reconocimiento de red o para fines educativos. Su razón de ser es profundizar en el manejo de sockets, el funcionamiento de los protocolos de red y la interacción con el sistema operativo desde C++, así como ejercitar conceptos de concurrencia (hilos, sincronización, escritura concurrente en memoria) propios de la materia de Sistemas Operativos. El resultado es una utilidad práctica que sirve como base para futuros desarrollos de seguridad y automatización de tareas de reconocimiento.

2. Tiempo estimado de desarrollo

El tiempo total estimado para el desarrollo completo del proyecto se situó entre 40 y 60 horas a lo largo del semestre, distribuidas de la siguiente manera:

- Análisis y diseño: 8–10 horas.
- Implementación del núcleo de red (sockets, escaneo básico): 15–20 horas.
- Funcionalidades avanzadas (concurrencia, timeouts, FILTERED, banners, rangos de IP, exportación CSV): 10–15 horas.
- Pruebas, validación y documentación: 8–10 horas.

3. Tecnologías usadas

3.1 Lenguaje de programación

C++ estándar C++17 o superior, ejecutándose sobre un sistema macOS.

3.2 Librerías

- **Biblioteca estándar de C++ (STL):** uso de `std::string`, `std::vector`, `std::map`, `std::thread`, `std::mutex`, `std::atomic`, `std::chrono`, `std::ofstream` y manejo básico de excepciones.

- **API de sockets Berkeley (POSIX):** llamadas como `socket()`, `connect()`, `select()`, `getaddrinfo()`, `setsockopt()`, `getsockopt()`, `fcntl()`, `inet_pton()`, `recv()`, `send()` y `close()` para la comunicación de red de bajo nivel y para la implementación del timeout configurable.

3.3 Herramientas extra

- **Compilador:** g++ o clang++.
- **Sistema operativo base:** macOS.
- **Sistema de construcción:** Makefile.
- **Control de versiones:** git + repositorio remoto en GitHub.
- **Herramientas de prueba y monitoreo (solo para validación):** nmap, ping, tcpdump y/o Wireshark.

4. Justificación tecnológica

Para el desarrollo de este escáner de red se utiliza C++ debido a su buen equilibrio entre rendimiento, control de bajo nivel y soporte de abstracciones de alto nivel mediante la biblioteca estándar. La API de sockets POSIX se elige porque expone directamente la interfaz de red que provee el sistema operativo, lo que permite entender de manera más fiel cómo se crean, gestionan y cierran conexiones TCP desde el espacio de usuario. Esto resulta fundamental en un escáner de puertos, donde es necesario controlar tiempos de espera, interpretar códigos de error y manejar múltiples conexiones concurrentes.

El uso de la biblioteca estándar de C++ permite manejar colecciones de puertos, rangos de direcciones IP y cadenas de texto de forma segura y eficiente, sin agregar dependencias externas de alto nivel que oculten el funcionamiento interno. Las herramientas de apoyo como git y GitHub facilitan el control de versiones y la colaboración, mientras que nmap y las herramientas de análisis de tráfico se utilizan únicamente como referencia para validar el comportamiento del escáner desarrollado.

5. Problemática que resuelve y público objetivo

El proyecto resuelve la necesidad de contar con una herramienta ligera y sencilla para reconocimiento de red y detección de puertos abiertos en hosts específicos o en rangos pequeños de direcciones. Este tipo de herramienta es útil en tareas de administración de sistemas, auditorías básicas de seguridad y prácticas de laboratorio en cursos de redes y ciberseguridad.

El escáner está dirigido a:

- Estudiantes de informática, ingeniería de software, redes y ciberseguridad.
- Administradores de sistemas que requieran una herramienta pequeña para pruebas rápidas.

- Personas interesadas en entender cómo funcionan internamente los escáneres de puertos.

La razón de ser del proyecto es doble: por un lado, aprender y practicar el uso de C++ y sockets en un entorno macOS, ejercitando además conceptos centrales de sistemas operativos como hilos, sincronización y E/S asíncrona; por otro, obtener una herramienta funcional que pueda utilizarse en ejercicios de reconocimiento de red y que sirva como base para futuras extensiones (nuevos tipos de escaneo, análisis de servicios, automatización).

6. Razón de uso de las librerías

Se usa la API de sockets POSIX para resolver la problemática de establecer conexiones TCP a múltiples puertos y hosts, ya que esta interfaz permite configurar timeouts, manejar estados de conexión y diferenciar entre distintos tipos de error (rechazo, timeout, fallo de resolución, etc.). Esto es clave para clasificar correctamente el estado de los puertos escaneados como OPEN, CLOSED o FILTERED.

La biblioteca estándar de C++ se utiliza para representar las listas de puertos, rangos de IPs y resultados del escaneo, así como para gestionar hilos de ejecución y sincronización en la fase de concurrencia, y para escribir el archivo CSV de exportación. Esta combinación ofrece suficiente control de bajo nivel para entender qué ocurre en el sistema operativo, pero con herramientas que facilitan la escritura de código mantenible y relativamente seguro.

7. Desarrollo del proyecto

7.1 Análisis y diseño

Para la entrega final se conservó la arquitectura de cuatro clases consolidada en el segundo parcial y se agregaron las cuatro funcionalidades que quedaron explícitamente planeadas para esta entrega: timeouts configurables, distinción entre los estados CLOSED y FILTERED, lectura opcional de banners y soporte de rangos de IP. Adicionalmente, se sumó una utilidad menor (exportación de resultados a CSV) sugerida ya en el plan original del proyecto.

La arquitectura quedó descompuesta en cuatro clases con responsabilidades separadas, exactamente como en el parcial anterior:

- **ScannerApp** orquesta el flujo general del programa.
- **ArgumentParser** interpreta y valida la línea de comandos.
- **PortScanner** ejecuta el escaneo concurrente sobre sockets TCP.
- **ResultFormatter** imprime la tabla de resultados y, en su caso, escribe el archivo CSV de exportación.

Los tipos compartidos (Args, PortResult, PortState, HostResults) se mantienen aislados en un encabezado Types.hpp para desacoplar las clases entre sí. Frente al parcial anterior, PortState añade el valor FILTERED, PortResult añade el campo banner, Args se extiende con

timeoutMs, readBanner, csvPath y un `std::vector<std::string>` de hosts en lugar de un único host, y se introduce el agregado `HostResults` para encapsular los resultados por host al exportar a CSV.

7.2 Implementación del núcleo de red

El núcleo de red reside en `PortScanner.cpp`. El flujo comienza con una única resolución del host mediante `getaddrinfo()`, de la cual se obtiene una estructura `sockaddr_in` base que contiene la dirección IPv4 resuelta. Esta estructura se copia (no se comparte por referencia) a cada hilo, de modo que cada hilo puede modificar independientemente el campo `sin_port` sin incurrir en una condición de carrera sobre la dirección de destino.

A diferencia del segundo parcial, donde `connect()` se ejecutaba en modo bloqueante con el timeout por defecto del sistema operativo, en la entrega final el socket se pone explícitamente en modo no bloqueante mediante `fcntl(sock, F_SETFL, O_NONBLOCK)`. Esto permite controlar de forma directa el tiempo máximo que se está dispuesto a esperar por puerto. La secuencia por puerto es ahora:

1. `socket(AF_INET, SOCK_STREAM, 0)` — crea un socket TCP IPv4.
2. `fcntl(sock, F_SETFL, O_NONBLOCK)` — activa modo no bloqueante.
3. `connect(sock, ...)` — inicia el three-way handshake. Como el socket es no bloqueante, retorna inmediatamente con `errno == EINPROGRESS` (caso común), `0` (conexión instantánea, típico de loopback) o `ECONNREFUSED` (rechazo activo del extremo remoto).
4. Si la conexión quedó en curso, se invoca `select()` sobre el conjunto de escritura del socket con un `timeval` correspondiente al timeout solicitado.
5. Clasificación del resultado:
 - `connect()` devuelve `0` o `select()` indica el socket listo y `SO_ERROR` es `0` → **OPEN**.
 - `errno == ECONNREFUSED` (directo) o `SO_ERROR == ECONNREFUSED` (tras `select()`) → **CLOSED**.
 - `select()` retorna `0` (timeout) o cualquier otro error tras el handshake → **FILTERED**.
6. `close(sock)` — libera el descriptor de archivo asociado al socket.

El uso de `select()` para acotar `connect()` resuelve la limitación principal documentada en el segundo parcial: el tiempo de ejecución contra `scanme.nmap.org` cayó de aproximadamente 75 s a apenas un par de segundos, y los puertos que el firewall del host remoto descarta silenciosamente quedan ahora correctamente etiquetados como **FILTERED** en lugar de aparecer como **CLOSED**.

7.3 Implementación del escáner de puertos con concurrencia

El esquema de concurrencia es el mismo que se introdujo en el segundo parcial: se lanzan hasta 256 hilos trabajadores (constante `kMaxWorkers`) que se reparten dinámicamente los puertos a escanear mediante un contador atómico `std::atomic<size_t> next{0}`. Cada hilo

entra a un bucle infinito del que sale cuando `next.fetch_add(1)` devuelve un índice mayor o igual al total de puertos. Esta forma corresponde en la práctica a un *thread pool* con cola de trabajo implícita, donde la cola es simplemente el vector de puertos y el “cursor” es el contador atómico.

El vector de resultados se pre-dimensiona antes de lanzar cualquier hilo. Cada hilo escribe exclusivamente en el índice que le corresponde (`results[i]`), por lo que no existe contienda de datos sobre el contenedor: el estándar de C++ garantiza que escrituras a elementos distintos de un contenedor pre-dimensionado son independientes siempre y cuando no se invoque `resize`, `push_back` o similares durante la ejecución concurrente.

El único recurso verdaderamente compartido entre hilos es `std::cerr`, al que los hilos podrían escribir si `socket()` falla por algún motivo (por ejemplo, agotamiento de descriptores de archivo). Para evitar que las líneas de error se intercalen de forma ilegible, se protege `std::cerr` con un `std::mutex` global (`g_cerrMutex`) mediante `std::lock_guard`. No se usa `mutex` para `std::cout` porque la impresión final la realiza `ResultFormatter` una vez que todos los hilos han terminado (`join()`).

7.4 Manejo flexible de la especificación de puertos

Esta funcionalidad se introdujo en el segundo parcial y se conserva sin cambios en la entrega final. `ArgumentParser` acepta cuatro formatos intercambiables para la opción `--ports`:

- **Rango:** `--ports 20-80`, se expande al intervalo cerrado `[20, 80]`.
- **Lista:** `--ports 22,80,443`, se interpreta como un conjunto explícito.
- **Puerto único:** `--ports 80`, se trata como un conjunto de un solo elemento.
- **Todos:** `--ports all`, equivale a los 65 535 puertos válidos.

La expansión produce en todos los casos un `std::vector<int>` ordenado que es lo que `PortScanner` consume.

7.5 Timeouts configurables y estado FILTERED

La bandera nueva `--timeout <ms>` permite al usuario fijar el tiempo máximo que el escáner está dispuesto a esperar por puerto. El valor por defecto se fijó en 1000 ms, suficiente para redes locales y hosts remotos con latencia moderada. El rango aceptado va de 1 ms a 60 000 ms.

La distinción entre `CLOSED` y `FILTERED` se basa enteramente en cómo responde la pila TCP del extremo remoto:

- Un RST inmediato (manifestado como `errno == ECONNREFUSED` o `SO_ERROR == ECONNREFUSED` tras `select()`) indica que la máquina remota recibió el SYN pero no hay servicio escuchando ese puerto: el puerto está cerrado de forma activa. Se clasifica como **CLOSED**.

- La ausencia total de respuesta dentro del timeout (`select()` retorna 0) indica que algún elemento intermedio (típicamente un firewall) está descartando silenciosamente los paquetes SYN. El puerto puede o no estar realmente cerrado; lo que se sabe con certeza es que no se puede decidir. Se clasifica como **FILTERED**.

Esta diferencia es exactamente la misma que reporta `nmap` por defecto en su TCP connect scan, y resuelve la limitación más visible del segundo parcial, donde todos los puertos sin respuesta se reportaban como CLOSED.

7.6 Lectura opcional de banners

La bandera nueva `--banner` activa, sólo para puertos detectados como OPEN, una fase adicional de lectura del banner inicial del servicio. El flujo es:

1. Tras confirmar que el puerto está OPEN, el socket se vuelve a poner en modo bloqueante restaurando los flags previos.
2. Se aplica `setsockopt(SO_RCVTIMEO)` con un timeout de a lo más 1500 ms (o el `--timeout` del usuario si es menor), para no quedarse esperando indefinidamente si el servicio no envía nada.
3. Para puertos típicamente HTTP (80, 8000, 8080) el escáner envía un `HEAD / HTTP/1.0\r\n\r\n` antes de leer, porque HTTP no envía banner espontáneo. Para el resto de puertos se intenta `recv()` directo, lo que cubre servicios como SSH, FTP, SMTP, POP3 o IMAP, que envían su banner identificativo en el momento de aceptar la conexión.
4. Los bytes recibidos se procesan: se cortan al primer `\r` o `\n`, se filtran caracteres no imprimibles y se limita la longitud a 80 caracteres para mantener legible la salida en consola.

Pruebas contra `scanme.nmap.org` con `--banner` permiten identificar la versión exacta del demonio SSH (SSH-2.0-OpenSSH_6.6.1p1 Ubuntu-2ubuntu2.13) y la línea de estado del servidor HTTP (HTTP/1.1 200 OK). Esta es una identificación de servicio real, basada en el contenido que el propio servicio anuncia, y complementa (no sustituye) la tabla estática de servicios bien conocidos asociada al número de puerto.

7.7 Soporte de rangos de IP

La bandera nueva `--range <ip1-ip2>` permite escanear un rango de direcciones IPv4 en lugar de un único host. La especificación se valida con `inet_pton(AF_INET, ...)` y, una vez expandida, se almacena como un `std::vector<std::string>` dentro de `Args`. Para evitar abusos se fijó un máximo de 1024 IPs por rango; un intento de escanear, por ejemplo, una /16 completa (65 536 IPs) es rechazado en el parseo con un mensaje claro.

`ScannerApp` itera sobre cada host del vector, llama a `PortScanner::scan()` una vez por host, imprime la tabla parcial por host y al final reporta el tiempo total. La fase de descubrimiento de hosts mencionada en el plan original como opcional (ICMP ping previo) no se implementó por

separado: en la práctica, el propio escaneo TCP cumple la función al clasificar como FILTERED los hosts que no responden a ningún puerto.

7.8 Exportación de resultados a CSV

La bandera nueva `--export <archivo.csv>` activa la escritura de un archivo CSV con todas las filas de resultados acumuladas: host, puerto, estado (OPEN/CLOSED/FILTERED), servicio inferido por número de puerto y banner leído (si lo hubo). El banner se entrecomilla y las comillas internas se duplican siguiendo la convención RFC 4180, de modo que el archivo es procesable directamente por herramientas como Excel, Numbers o pandas.

La exportación se realiza al final de todas las fases de escaneo, no en paralelo con la salida de consola, para mantener un único punto de escritura a disco.

7.9 Presentación de resultados

ResultFormatter imprime exclusivamente los puertos detectados como OPEN, en una tabla alineada mediante `std::setw` y acompañada de las columnas ESTADO, SERVICIO y BANNER. La identificación heurística del servicio se apoya en un `std::map<int, std::string>` que asocia aproximadamente 17 puertos bien conocidos (SSH, HTTP, HTTPS, MySQL, PostgreSQL, FTP, SMTP, DNS, etc.) con sus nombres. Los puertos abiertos cuyo número no aparezca en el mapa se etiquetan como “desconocido”; con `--banner` activo, su banner real suele aportar la información de servicio que la tabla estática no tiene.

Al final del escaneo se imprime una línea de resumen con el total de puertos analizados, cuántos resultaron abiertos, cuántos cerrados, cuántos filtrados y el tiempo total transcurrido medido con `std::chrono::steady_clock`. Los puertos cerrados y filtrados deliberadamente no se imprimen uno por uno para evitar saturar la consola en rangos grandes; únicamente sus conteos aparecen en la línea de resumen.

Ejemplo de salida real para `./scanner --host scanme.nmap.org --ports 22,80,443 --timeout 3000 --banner:`

```
Escaneando scanme.nmap.org (3 puertos)...
```

PUERTO	ESTADO	SERVICIO	BANNER
22	OPEN	SSH	SSH-2.0-OpenSSH_6.6.1p1 Ubuntu-2ubuntu2.13
80	OPEN	HTTP	HTTP/1.1 200 OK

```
-----
Total: 3 puertos | Abiertos: 2 | Cerrados: 1 | Filtrados: 0
Tiempo: 0.23s
```

7.10 Pruebas y validación

Se realizaron las siguientes pruebas manuales para validar la corrección y el comportamiento del escáner en su versión final:

1. Localhost con distintos formatos de --ports. Se probaron los cuatro formatos: rango (20-80), lista (22,80,443), puerto único (80) y all. Los cuatro producen el formato de salida esperado y clasifican correctamente los puertos.

2. Host remoto con timeout configurable. Ejecutando `./scanner --host scanme.nmap.org --ports 20-100 --timeout 2000`, el escaneo de 81 puertos terminó en 2.15 s frente a los 75 s reportados en el segundo parcial cuando se dependía del timeout por defecto del sistema operativo. La diferencia, de más de un orden de magnitud, valida que `select()` está acotando efectivamente la espera por puerto.

3. Banner real contra scanme.nmap.org. Ejecutando `./scanner --host scanme.nmap.org --ports 22,80,443 --timeout 3000 --banner`, el escáner identificó correctamente la versión exacta del servidor SSH (OpenSSH_6.6.1p1 Ubuntu-2ubuntu2.13) y la línea de estado HTTP (HTTP/1.1 200 OK), demostrando que la fase de lectura de banners funciona tanto para servicios que envían banner espontáneo como para servicios que requieren un probe inicial.

4. Rango de IPs con estados mixtos. Ejecutando `./scanner --range 127.0.0.1-127.0.0.3 --ports 80,443,3306 --timeout 500`, 127.0.0.1 reporta los tres puertos como CLOSED (RST inmediato del kernel local, ya que no hay servicio escuchando) mientras que 127.0.0.2 y 127.0.0.3 reportan los tres puertos como FILTERED (la dirección no responde y se llega al timeout). Esto valida en una sola corrida la distinción CLOSED/FILTERED y el escaneo multi-host.

5. Exportación CSV. Ejecutando el mismo escaneo con `--export resultados.csv`, el archivo generado contiene una fila por puerto con las columnas host, puerto, estado, servicio, banner, con los banners entrecomillados y las filas sin banner con campo vacío. El archivo se abre correctamente en herramientas estándar de hojas de cálculo.

6. Entradas inválidas. Se probaron múltiples casos degenerados: ejecución sin argumentos, `--timeout abc`, `--range` con `ip1 > ip2`, `--range` con más de 1024 IPs, especificar simultáneamente `--host` y `--range`, argumentos desconocidos y un host no resoluble por DNS. Todos los casos emiten un mensaje de error claro y terminan con código de salida distinto de cero, sin llegar nunca a la fase de escaneo (excepto el host no resoluble, que se detecta dentro de `PortScanner::scan` y produce un mensaje de error sin abortar el resto de hosts del rango si los hubiera).

7. Comparación con nmap. Para `scanme.nmap.org`, tanto `nmap -p 22,80 scanme.nmap.org` como `./scanner --host scanme.nmap.org --ports 22,80 --timeout 2000` reportan los dos puertos como OPEN/abiertos. Para rangos donde nmap reporta algunos puertos como filtered, el escáner desarrollado también los reporta como FILTERED siempre que el `--timeout` sea suficientemente amplio para no truncar respuestas tardías legítimas.

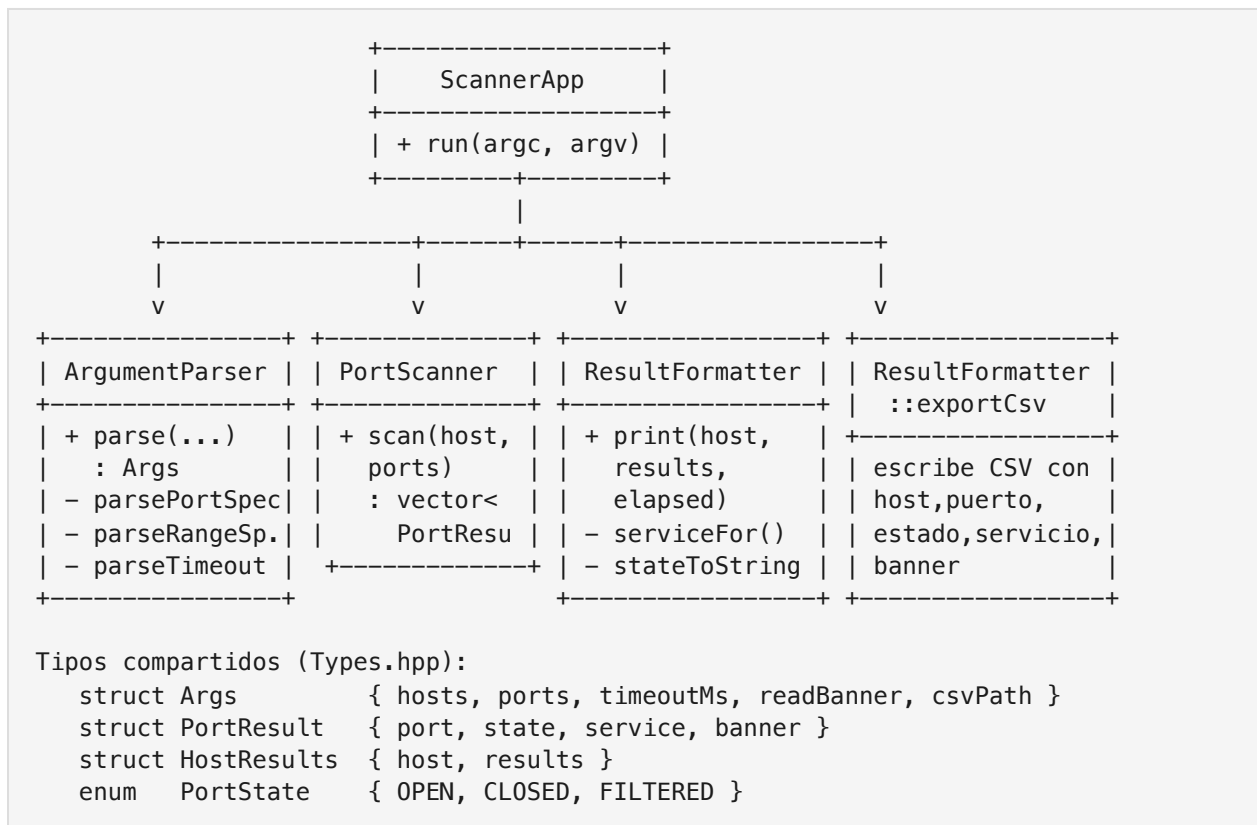
Limitaciones conocidas (residuales)

Se conservan las siguientes limitaciones, que quedan fuera del alcance del proyecto:

- Sólo se implementa **TCP connect scan**. Escaneos avanzados como SYN scan, UDP scan o scripts NSE quedan fuera del alcance, ya que requieren sockets raw y privilegios de administrador.
- El descubrimiento de hosts activos previo al escaneo (ICMP ping) no se implementó como fase separada; el propio escaneo TCP cumple la función al marcar los hosts no responsivos como FILTERED en todos sus puertos.
- La heurística de banner no implementa fingerprinting profundo: se limita a leer los primeros bytes y, para HTTP, a enviar un único probe HEAD. Servicios sobre TLS (HTTPS, IMAPS, etc.) no producen banner legible porque su primer mensaje es un ClientHello cifrado.

8. Diagramas

8.1 Diagrama de clases

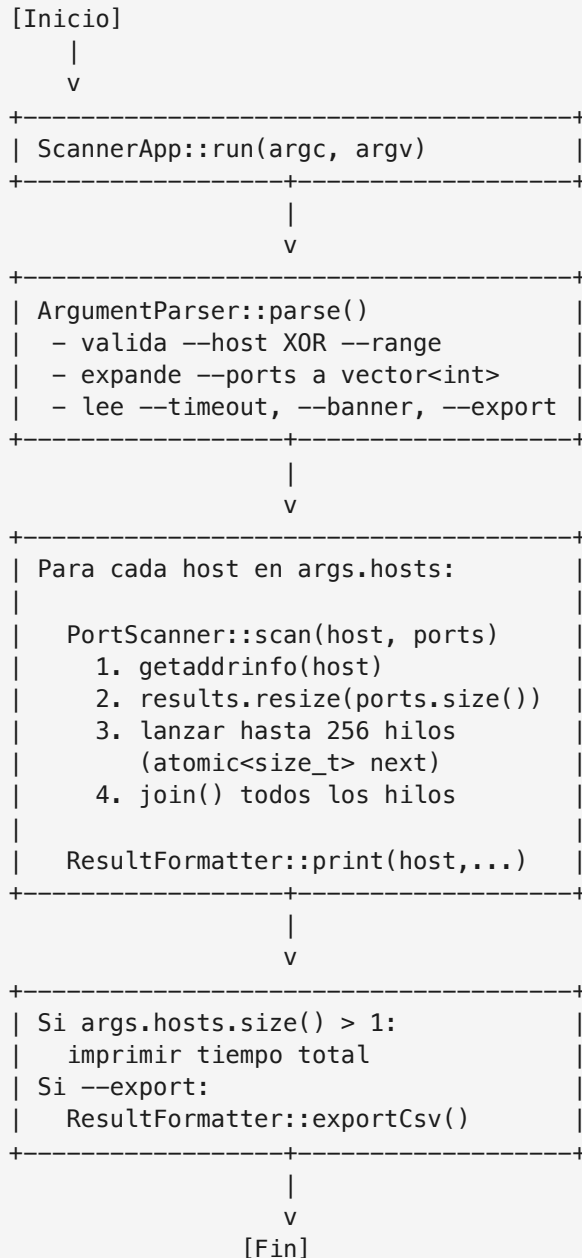


Responsabilidades por clase:

- **ScannerApp:** orquesta la ejecución del programa. No contiene lógica de negocio; únicamente coordina las llamadas a las demás clases, itera sobre `args.hosts`, mide el tiempo total y, al final, dispara la exportación CSV si fue solicitada.
- **ArgumentParser:** expone métodos estáticos puros. Convierte `argv` en una estructura `Args` validada. Ante error de parseo, termina el proceso con un mensaje claro al usuario.

- **PortScanner:** contiene la lógica de sockets y concurrencia. Es la única clase que interactúa con el sistema operativo a nivel de red (llamadas POSIX). Recibe el timeout y la bandera de banner en el constructor y los aplica internamente a cada hilo.
- **ResultFormatter:** expone métodos estáticos puros. Convierte un vector<PortResult> en salida formateada sobre `std::cout` y, por separado, convierte un vector<HostResults> en un archivo CSV.

8.2 Diagrama de flujo del programa



Detalle del cuerpo de cada hilo (scanSinglePort):

```

[hilo creado]
|
v
socket() --> fcntl(0_NONBLOCK)
|
v
sin_port = htons(port)
|
v
connect() --> ¿ret == 0?
| sí \ no, ¿EINPROGRESS?
| v \
state = OPEN v | no, ¿ECONNREFUSED?
| select(timeout) v |
| / | \ state=CLOSED | otro
| / | \ | v
| ret=0 listo+S0 error/ state=FILTERED
| (timeout) =0/REFUSED fallo |
| v v v v v
state=FILTERED state=OPEN state=CLOSED/FILTERED
|
v
si state == OPEN y --banner:
    fcntl(restaurar flags)
    setsockopt(SO_RCVTIMEO)
    probe HTTP si port∈{80,8000,8080}
    recv() y limpiar caracteres
|
v
close(sock)
[fin hilo]

```

9. Justificación técnica

C++17 y STL. Se eligió C++17 por su soporte nativo de `std::thread`, `std::mutex`, `std::lock_guard`, `std::atomic`, y por la posibilidad de usar capturas de lambda con inicialización explícita (`[&, baseTarget, timeoutMs, wantBanner]`) que simplifican notablemente el paso de parámetros a los hilos. Alternativas como el uso directo de `pthread` introducirían más código repetitivo sin aportar valor pedagógico adicional en este alcance.

Sockets POSIX. El uso directo de `socket()`, `connect()`, `select()`, `getsockopt()`, `setsockopt()`, `fcntl()`, `getaddrinfo()`, `recv()`, `send()` y `close()` es deliberado: expone al estudiante al mismo contrato que el kernel ofrece a cualquier proceso, sin esconderlo detrás de una biblioteca de alto nivel. Esto es didácticamente importante en un curso de Sistemas Operativos, ya que permite razonar directamente sobre descriptores de archivo, estados del socket, modos bloqueante/no bloqueante y códigos de error de la llamada al sistema.

Connect no bloqueante + select() en lugar de setsockopt(SO_SNDTIMEO). En macOS, `SO_SNDTIMEO` no acota de forma fiable la duración de `connect()` sobre sockets bloqueantes en todas las versiones del sistema. La forma canónica y portable de imponer un timeout a `connect()` es la usada en este proyecto: poner el socket en `O_NONBLOCK`, lanzar el `connect()`, y esperar sobre el conjunto de escritura con `select()`. Esta es la misma técnica documentada por Stevens en *UNIX Network Programming* y la que utiliza `nmap` internamente en su TCP connect scan.

Modelo de concurrencia con tope de 256 hilos y cola implícita. Entre las tres alternativas consideradas — (a) escaneo secuencial, (b) un hilo por puerto, (c) thread pool con cola de trabajo compartida — se eligió un diseño intermedio: hasta 256 hilos que se reparten dinámicamente los puertos mediante un contador atómico. Esto introduce concurrencia real y permite demostrar explícitamente la sincronización con `std::mutex` sobre `std::cerr`, sin requerir las estructuras de un thread pool clásico (`std::condition_variable`, cola protegida por mutex, lógica de terminación con poison pill). El modelo escala razonablemente hasta aproximadamente 65 535 puertos en macOS, suficiente para los rangos de prueba contemplados.

Ausencia de carrera de datos en results. Aunque múltiples hilos escriben sobre el mismo `std::vector<PortResult>`, cada hilo escribe exclusivamente en `results[i]` con un `i` único por hilo (asignado por el contador atómico). El estándar de C++ (§23.2.2 `container.requirements.dataraces`) permite escrituras concurrentes a elementos distintos de un contenedor siempre y cuando no se modifique la estructura del contenedor durante las operaciones (es decir, sin `push_back`, `resize`, etc.). Pre-dimensionar el vector con `results(ports.size())` antes de lanzar los hilos garantiza esta precondition.

Mutex únicamente para std::cerr. La única sincronización explícita necesaria es proteger las escrituras a `std::cerr` desde múltiples hilos ante un fallo de `socket()`. `std::cout` no se utiliza dentro de los hilos, sino sólo desde `ResultFormatter::print()` una vez que todos los hilos han sido unidos (`join()`), por lo que no necesita protección alguna. Esta asimetría ilustra un principio general: sincronizar únicamente los recursos realmente compartidos, no “por si acaso”.

Banner sólo bajo bandera explícita. La lectura de banner se mantiene desactivada por defecto. La justificación es triple: (a) introduce latencia adicional por cada conexión abierta, (b) genera tráfico en el extremo remoto que puede no ser deseable en una auditoría discreta, y (c) algunos servicios pueden cerrarse al detectar tráfico inesperado en el primer byte. Mantenerla opt-in respeta el principio de mínima sorpresa.

Tabla estática de servicios + banner real, en complemento. Se conserva el `std::map<int, std::string>` con puertos bien conocidos como identificación heurística rápida sin coste de red, y se complementa con la lectura real del banner sólo cuando el usuario lo pide. Ambas identificaciones se imprimen lado a lado en la tabla de resultados: la columna `SERVICIO` viene de la tabla estática, la columna `BANNER` viene del propio servicio.

10. Entregas por parcial (planificación)

Primer parcial (25 de febrero de 2026) — entregado.

- Un tercio del reporte escrito: introducción, resumen, objetivos, tecnologías, justificación tecnológica, problemática y público objetivo.
- Un tercio de la funcionalidad: programa que recibe una IP y un rango pequeño de puertos y realiza un escaneo básico sin concurrencia.
- Repositorio en GitHub creado con el proyecto inicial.

Segundo parcial (22 de abril de 2026) — entregado.

- Otro tercio del reporte escrito: sección de desarrollo completa, diagramas de clases y de flujo, justificación técnica detallada de las decisiones de concurrencia.
- Otro tercio de funcionalidad: manejo flexible de rangos de puertos (rango, lista, puerto único, `all`), formato de salida mejorado con tabla de servicios y resumen, y concurrencia mediante hasta 256 hilos con sincronización sobre `std::cerr`.
- Repositorio GitHub actualizado con la arquitectura de cuatro clases.

Final de semestre — entrega actual.

- Reporte completo en PDF con nombre en formato `ID_Nombre_ApellidoPaterno`.
- Proyecto funcionando con todas las funcionalidades definidas:
 - Timeouts configurables (`--timeout <ms>`) implementados con `connect()` no bloqueante + `select()`.
 - Distinción real entre puertos CLOSED (RST inmediato) y FILTERED (timeout sin respuesta).
 - Lectura opcional de banners (`--banner`) con probe HTTP automático para puertos web.
 - Soporte de rangos de IPv4 (`--range ip1-ip2`) con cota de 1024 IPs.
 - Exportación de resultados a CSV (`--export <archivo>`).
- Repositorio GitHub completo y actualizado.